Projet de Programmation
Master Informatique
2024–2025

université de BORDEAUX

Le jeu de Quoridor

Cahier des charges

7 janvier 2025
Version 0.1

Ce projet a pour objectif de développer un logiciel permettant de jouer à Quoridor avec deux, trois ou quatre utilisateurs. Le programme doit permettre aux joueurs de s'affronter sur un plateau de jeu en respectant les règles.

Mots-clefs: Deux à quatre joueurs, déterministe, information parfaite.

1 Généralités

F1. Langage de programmation

Le langage de programmation de ce projet doit être choisi parmi les langages suivants :

- **Langage C** : C11 (ISO/IEC 9899 :2011).
- **Langage Java** : Java SE 17+.
- **Langage Python** : Python 3.7+.

F2. Style de codage

Le *coding style* du programme doit respecter :

- **Langage C** : le *coding style* de GNU.
- **Langage Java** : le *coding style* de Google.
- **Langage Python** : le *coding style* du PEP8.

F3. Langue par défaut dans le code

La documentation, le nom des variables, des fonctions et des fichiers devront être en anglais.

F4. Système cible

Le programme devra fonctionner sur des systèmes d'exploitation GNU/Linux.

F5. Documentation

Le programme et le code source devra être documenté : manuel utilisateur, option d'aide en ligne de commande et commentaires dans le code source.

F6. Tests

Le programme devra être testé et validé par suffisamment de tests pour garantir son bon fonctionnement (couverture d'au moins 80%). Les tests doivent être automatisés et facile à lancer.

F7. Bugs

Aucun bug menant à un crash ne doit être présent dans le programme. Les spécifications doivent être respectées et les erreurs doivent générer des messages d'erreur.

F8. Performances

Les performances du programme doivent être mesurées et documentées pour différentes entrées du programme ainsi que pour ses cas limites.

2 Bibliothèques et outils

F9. Build-system

Le '*build-system*' devra être basé sur :

- **Langage C** : `CMake`.
- **Langage Java** : `Maven`.
- **Langage Python** : `pyproject.toml + setuptools`.

F10. Frameworks de tests

Le *framework* de test sera basé sur :

- **Langage C** : `CTest + googletest + gcovr`.
- **Langage Java** : `JUnit + Jacoco`.
- **Langage Python** : `pytest + coverage`.

F11. Gestion des options

Les options en ligne de commande seront gérées par :

- **Langage C** : le module `getopt_long`.
- **Langage Java** : la bibliothèque `commons-cli`.
- **Langage Python** : le module `argparse`.

F12. Bibliothèque graphique

La bibliothèque graphique utilisé sera basé sur :

- **Langage C** : LibGtk 4.x.
- **Langage Java** : JavaFx.
- **Langage Python** : PyGObject.

F13. Internationalisation

Le *framework* d'internationalisation sera basé sur :

- **Langage C** : la bibliothèque `gettext`.
- **Langage Java** : `ResourcesBundle` et `Locale`.
- **Langage Python** : le module Python `gettext`.

La langue par défaut, sans locale précisée, sera l'anglais et, au moins, le français sera ajouté.

3 Options en ligne de commande

F14. Nom de l'exécutable principal

L'exécutable principal doit s'appeler '`quoridor`' et permettre de lancer tout le programme.

F15. Usage général

Le programme sera lancé comme ceci :

```
#> quoridor [OPTIONS] [FILENAME]
```

Si une option est incorrecte, le programme devra afficher un message d'erreur et l'aide du programme.

Si un nom de fichier est donné, le programme devra charger une partie depuis ce fichier. Sinon, le programme devra démarrer une nouvelle partie.

Les messages à l'utilisateur seront écrits sur la sortie standard (`stdout`) et les messages d'erreurs devront être écrits sur la sortie d'erreur standard (`stderr`). De plus, le code de retour du programme devra être `EXIT_SUCCESS` en cas de succès et `EXIT_FAILURE` en cas d'erreur.

F16. Aide en ligne de commande

L'option '`-h`', '`--help`' affiche l'aide puis quitte.

F17. Version

L'option '`-V`', '`--version`' affiche la version puis quitte.

F18. Mode `verbose`

L'option '`-v`', '`--verbose`' permettra d'afficher des informations supplémentaires pendant l'exécution.

F19. Mode `debug`

L'option '`-d`', '`--debug`' permettra d'afficher des informations de débogage pendant l'exécution.

F20. Nombre de joueurs

L'option '`-p N`', '`--players N`' permettra de spécifier le nombre de joueurs parmi 2, 3 ou 4. Par défaut, le nombre de joueurs sera de 2. Notez qu'il n'est pas conseillé de jouer en mode 3 joueurs car le joueur qui n'a pas d'adversaire direct a un avantage.

F21. Nombre de murs

L'option '`-w N`', '`--walls N`' permettra de spécifier le nombre de murs que chaque joueur possède. Par défaut, le nombre de murs sera de 20. Si `N` est négatif, le nombre de murs sera illimité.

F22. Taille du plateau

L'option '`-s N`', '`--size N`' permettra de spécifier la taille du plateau de jeu qui seront toujours des carrés. La taille devra être impaire et comprise entre 3 et 15, toute autre valeur entraînera une erreur. Par défaut, la taille du plateau sera de 9.

F23. Mode `blitz`

L'option '`-b`', '`--blitz`' permettra de jouer en mode blitz. Dans ce mode, un temps limite sera donné à chaque joueur pour jouer un coup. Si le temps d'un des joueur est écoulé, il perd la partie.

F24. Durée du blitz

L'option '`-t TIME`', '`--time TIME`' permettra de spécifier le temps limite en minutes pour chaque joueur en mode blitz. Par défaut, le temps limite sera de 15 minutes. Si cette option est donnée sans l'option '`-b`', '`--blitz`', elle sera ignorée mais un message d'avertissement sera affiché.

F25. Mode `contest`

L'option '`-c FILENAME`', '`--contest FILENAME`' permet de lancer le programme en mode '`contest`' en chargeant une partie depuis un fichier '`FILENAME`'. Le programme devra alors afficher le joueur et le coup à jouer au format décrit dans cette spécification.

F26. Mode IA

L'option '`-a[COLOR]`', '`--ai[=COLOR]`' permet de lancer le programme en mode joueur artificiel avec la couleur `COLOR` ('`1`', '`2`', ...et '`A`' (*All*), `2` par défaut). Cette option est cumulative, par exemple '`-a -a4`' lancera trois IA de couleur 2, 3 et 4. Notez aussi que si l'IA de couleur 4 est lancée, le nombre de joueurs sera automatiquement fixé à 4 même sans recours à l'option '`-p4`' et le programme devra en déduire que le joueur 1, 2 et 3 sont humains. De plus, en cas de conflit entre l'option '`-a`' et l'option '`-p`', l'option '`-p`' sera ignorée mais un message d'avertissement sera affiché sur `stderr`.

4 Interface utilisateur

F27. Interface en ligne de commande

Le programme devra posséder une interface en ligne de commande qui permette une interaction avec l'utilisateur via le terminal. Cette interface textuelle sera lancée par défaut par le programme.

F28. Interface graphique

Le programme devra posséder une interface graphique qui permette les mêmes interactions que sur le CLI avec l'utilisateur via une fenêtre graphique. Cette interface graphique sera lancée si l'option '`-g`', '`--gui`' est donnée.

F29. Affichage du plateau

À chaque coup, le programme devra afficher le plateau ainsi que le temps pris par chaque joueur si la partie est en blitz.

En mode ASCII le pion de chaque joueur sera représenté par le numéro du joueur (1, 2, 3, 4). Les cases vides du plateau seront représentées par un '`_`' et les murs par des '`X`', enfin les espaces verticaux entre deux cases vides par des '`.`'.

F30. Notation des coups

Le programme devra permettre de déplacer les pions en indiquant la position de départ et la position d'arrivée. Les positions seront indiquées en numérotant chaque case à partir de sa colonne (de 'a' à 'i' en partant de la gauche) et de sa ligne (de 1 à 9 en partant du haut). Par exemple, le coup '`e2-e3`' déplace le pion de la case 5 à la case 6. Pour placer un mur, on prendra la case qui est en haut à gauche du mur et on indiquera la direction du mur. Par exemple, le coup '`e2v`' place un mur vertical à droite de la case `e2` alors que le coup '`e2h`' place un mur horizontal au-dessous de la case `e2`.

F31. Affichage de l'historique

L'historique sera représenté comme une liste des tours du jeu. Par exemple, le premier tour sera affiché comme suit : '`1. 1 e2-e3 2 e3v`' (le numéro du tour dans la partie, le joueur puis la notation des coups de chaque joueur à ce tour).

F32. Abandon

Le programme devra permettre à un joueur d'abandonner la partie en cours de jeu.

F33. Affichage des messages

Le programme devra afficher des messages d'information pendant la partie en fonction du mode dans lequel il est (`default`, `verbose`, `debug`).

F34. Quitter et sauvegarde une partie

Le programme devra permettre de quitter une partie en cours et de la sauvegarder dans un fichier.

F35. Sauvegarde de l'historique

Le programme devra pouvoir sauvegarder l'historique des coups joués dans un fichier à la demande de l'utilisateur.

F36. Naviguer dans l'historique

Il doit être possible de revenir en arrière ou d'aller en avant dans l'historique des coups joués si tous les joueurs humains sont d'accord.

F37. Recommencer une partie

À tout moment, il doit être possible de recommencer une partie depuis le début en abandonnant la partie en cours.

F38. Fin de partie

Le programme doit détecter la fin de la partie et afficher le résultat de la partie ainsi que le gagnant s'il y en a un.

F39. Rejouer une partie

Il doit être possible de rejouer une partie depuis le début en chargeant un fichier d'historique. Il doit être possible d'avancer et de reculer dans l'historique des coups voire de recommencer une partie depuis une position choisie par l'utilisateur (mais l'historique de la partie initiale au-delà de la position choisie par l'utilisateur sera perdu).

Par défaut, le programme affichera initialement l'état qui se situe à la fin de l'historique après son chargement.

5 Formats d'entrée/sortie

F40. Format de fichier simplifié

Le programme devra être capable de sauvegarder et restaurer des fichiers dans un format ASCII simplifié qui représente un plateau de jeu. Par exemple, un fichier de sauvegarde pourrait ressembler à ceci :

```
# Commentaire
2 # Couleur du joueur courant
# Puis le plateau de jeu (2 joueurs)
- - - - - - - - -
. . . . . . . . .
- - - -  1  - - - -
. . . . X . . . .
- - - - - - - - -
. . . . . . . . .
- - - - - - - - -
. . . . . . . . .
- - - - - - - - -
. . . . . . . . .
- - - - - - - - -
. . . . . . . . .
- - - - - - - - -
. . . . . . . . .
- - - - - - - - -
. . . . . . . . .
- - - -  X2 - - - -

# En option, l'historique des coups joués
```

Une erreur de lecture dans un fichier devra être signalée par une erreur à l'utilisateur en spécifiant le type d'erreur et la ligne où elle a été trouvée et le programme devra s'arrêter.

F41. Format de l'historique

Le programme devra être capable de lire et d'écrire des historiques de coups joués. Ils se présenteront comme une liste de tours avec les coups de chacun des joueurs dans l'ordre, par exemple :

```
# Commentaire
# Puis les tours joués
1. 1 e1-e2 2 a10-a9 3 e2-e3 4 a9v
2. 1 a6v 2 a9-a8 3 e4-e5 4 a8-a7
...
```

F42. Fichier de configuration

Le programme devra être capable de lire un fichier de configuration (`.quoridorrc`) qui contiendra les options par défaut du programme. Ce fichier sera lu au démarrage du programme et les options qu'il contient seront utilisées par défaut. Les options passées en ligne de commande devront écraser les options du fichier de configuration. On utilisera le format INI pour ce fichier, par exemple :

```
[Default]
players = 2
blitz = false
time = 30
ai = None
ai-mode = minimax
ai-depth = 3
ai-heuristic = default
ai-time = 5
ai-level = medium
```

6 Structure internes

F43. Structures de données

Le plateau de jeu sera représenté par un graphe non orienté. On utilisera une liste d'adjacence pour représenter les liens entre les cases du plateau. Poser un mur revient à supprimer une arête du graphe. La position des pions des joueurs sera stockée à part dans la structure qui représentera l'état du jeu.

F44. Module `graph`

Les listes d'adjacence seront stockées dans une table de hachage et on implémentera un certain nombre de fonctions utilitaires qui permettront de manipuler cette structure de donnée (création, destruction, recherche, insertion ou suppression d'un arc, …).

Le module `graph` devra aussi comporter une fonction capable de trouver le plus court chemin entre n'importe quel nœud et l'un des bords du plateau (designés par `N` (Nord), `S` (Sud), `E` (Est) et `W` (Ouest)).

F45. État du jeu

L'état du jeu sera représenté par une structure de données qui contiendra au moins :

- Le joueur courant.
- Les temps restants pour chaque joueur si la partie est en mode blitz.
- L'état du plateau de jeu.
- L'historique des coups joués.

7 Interface graphique

F46. Fonctions de base

Toutes les fonctions demandées pour l'interface en console devront être implémentées dans l'interface graphique.

F47. Menu 'File'

L'interface graphique devra comporter un menu principal qui permettra de lancer une nouvelle partie ('New Game'), de charger une partie depuis un fichier ('Load Game'), de sauvegarder la partie en cours dans un fichier ('Save Game'), de quitter le programme ('Quit'), d'afficher l'aide ('Help') ou d'éditer la configuration par défaut ('Settings').

F48. Menu 'Game'

L'interface graphique devra comporter un menu de jeu qui permettra de demander un conseil ('Hint'), naviguer dans l'historique (revenir en arrière, aller en avant, etc).

F49. Menu 'About'

L'interface graphique devra comporter un menu d'options qui permettra de consulter des informations sur le programme (version, auteurs, ...).

F50. Affichage d'une partie en cours

L'interface graphique devra afficher le plateau de la partie en cours ainsi que les informations habituellement présentes sur l'interface en ligne de commandes.

F51. Affichage des coups joués

L'interface graphique devra afficher sur un côté de l'interface, les coups joués lors de la partie en les numérotant.

F52. Configuration d'une nouvelle partie

L'item 'New Game' du menu 'File' permettra de mener l'utilisateur à une fenêtre de configuration de la partie qui sera déjà remplie avec les paramètres par défaut. Qu'ils soient modifiés ou non, l'utilisateur pourra lancer la partie en cliquant sur 'Start'.

8 Joueur artificiel

F53. Heuristique d'évaluation de position

Le programme devra comporter au moins une fonction capable de prendre un état du jeu et de lui donner un score en fonction de différents critères qui vous sembleront pertinents.

F54. Algorithme de recherche Minimax

Le choix du meilleur coup se fera via une exploration de l'arbre des coups possibles via l'algorithme de recherche Minimax. Cette option sera activée par défaut si le programme est lancé en mode joueur artificiel.

F55. $\alpha\beta$-pruning

Le choix du meilleur coup se fera via une exploration de l'arbre des coups possibles via l'algorithme de l'$\alpha\beta$-pruning. Cette option sera activée par défaut si le programme est lancé en mode joueur artificiel. Néanmoins, l'option '`--ai-mode ab`' sera tout de même disponible.

F56. Profondeur de recherche

L'option '`--ai-depth DEPTH`' permettra de spécifier la profondeur de recherche de l'algorithme de recherche.

F57. Choix des heuristiques

Il doit être possible de gérer plusieurs heuristiques d'évaluation de position. L'option '`--ai-heuristic HEURISTIC`' permettra de spécifier l'heuristique choisie.

F58. Heuristique de fin de partie

Créer au moins une heurististe de fin de partie pour optimiser l'évaluation des positions.

F59. Temps de réflexion borné

L'option '`--ai-time TIME`' permettra de spécifier le temps de réflexion de l'algorithme de recherche. Si le temps est écoulé, le programme devra jouer le meilleur coup trouvé jusqu'à présent. Par défaut, le temps de réflexion sera de 5 secondes.

F60. Classement des heuristiques

On étudiera les différentes heuristiques du jeu et on les classera en fonction de leur efficacité à remporter des parties et du nombre de joueurs (2, 3, 4).

Références

[1] Lisa Glendenning. *Mastering Quoridor*. Bachelor thesis, Department of Computer Science, University of New Mexico, 2005.

[2] P. J. C. Mertens. *A Quoridor-playing Agent*. Bachelor thesis, Department of Computer Science, Maastricht University, June 2006.